ArenaBook Client Setup Guide: `champions`

Here's the full flow, using a new client called `champions` as the example.

1. Create the client (once per client)

1a. Pick an id. Use lowercase letters, digits or `_`, for example `champions`. It becomes the flavor name, the end of the app id (`com.devcatstudios.arenabook.champions`) and the client's key in the Gist. Never change it after the first release.

1b. Generate the client:

```
dart run tool/clients.dart add champions
```

This creates `lib/app/config/clients/champions.dart` (copied from demo) and adds the client everywhere else it must be listed: Android, iOS, the Podfile and VS Code.

1c. Edit the client's settings in `lib/app/config/clients/champions.dart`: - sports and courts - rates - currency - countryCode - enableDiscount

Keep the sport and court ids stable once bookings exist.

1d. Client images (optional):

```
# put the files directly in assets/clients/champions/  (e.g. logo.png)
dart run tool/clients.dart assets champions
```

In code, use `Image.asset(AppClientConfig.client.asset('logo.png'))`.

1e. Launcher icon (optional; without one the client gets the ArenaBook icon): - **Android:** copy the icon files into `android/app/src/champions/res/`, using the same folders and names as `src/main/res`. Nothing else to do. - **iOS:** add `ios/Runner/Assets.xcassets/AppIcon-champions.appiconset`, then run:

```
dart run tool/clients.dart ios-icon champions
```

1f. Trial switch: Add the client to the clients map in your Gist:

```
"champions": { "DataEntryStatus": 1, "IsAppEnable": true, "AppDisableAfterDate": ""
```

You can skip this; a client that isn't in the Gist simply runs with no restriction.

1g. Verify:

```
flutter test      # clients_config_test fails if anything is out of step
```

2. Run and debug

```
flutter run --flavor champions
```

In VS Code, pick `champions` or `champions (release)` in Run and Debug. Plain `flutter run` always runs demo.

3. Before each release (once per release, for all clients)

1. **Bump version:** in `pubspec.yaml`, e.g. `1.0.1+2`. The build number after `+` must go up with every store upload.
2. Keep `kappVersionWithDate` and `kappBuildNumber` in `lib/app/config/app_strings.dart` in step with it.
3. Run `flutter test`.

All clients share the codebase version, so a release is a rebuild of each client from the same code.

4. Build

Windows (Android only):

```
scripts\build.bat champions apk          :: APK to send or sideload
scripts\build.bat champions appbundle    :: .aab for Play Store
```

Mac (Android or iOS):

```
bash scripts/build.sh champions apk
bash scripts/build.sh champions appbundle
bash scripts/build.sh champions ipa      # Mac only
```

Output: `dist/ArenaBook-champions-<version>.apk / .aab / .ipa`. Any extra arguments go straight to `flutter build`, for example:

```
scripts\build.bat champions apk --obfuscate --split-debug-info=build/symbols
```

To release every client, run the script once per client: `demo`, `champions`, and so on.

Not ready yet (affects store uploads)

- **Play Store:** release builds are still signed with the debug key. The APK installs fine on a phone, but Play Console will reject the `.aab`. We need an upload keystore first; this is the open "release signing" item.
- **iOS:**
- The first build on a Mac is the real test of the iOS setup.
- You must pick your Apple Team under **Runner → Signing & Capabilities**, so that `flutter build ipa` can sign.
- Each client's bundle id (`com.devcatstudios.arenabook.champions`) must be registered in your Apple Developer account.

Short version

```
dart run tool/clients.dart add champions     # 1. create
# edit lib/app/config/clients/champions.dart # 2. configure
dart run tool/clients.dart assets champions  # 3. images (optional)
```

```
flutter test                                    # 4. verify
scripts\build.bat champions apk                 # 5. build → dist/
```